

Atelier — Grand projet (preuve du sas)

À ouvrir après 1-COURS.pdf du dossier projets/09-grand-projet/.

1–3 périodes. Un mini-domaine à vous, une classe, une règle.

Le dossier depart/ de cet exercice contient les fichiers de départ. Vous n'avez pas à ouvrir un terminal.

Où l'on va

Vous avez déjà rencontré des classes et des objets. Ici, la preuve est de faire tenir cette idée dans **votre domaine** : une personne ouvre la page, comprend le sujet et voit une règle décidée par les objets.

Le départ utilise une fromagerie pour montrer la forme attendue. Vous ne collez pas cette fromagerie : vous changez les mots, les données et la règle. Le système solaire n'est pas un sujet possible et aucune animation d'orbites n'est demandée.

À la fin, votre écran a cette clarté, mais il raconte votre domaine. Voici seulement l'exemple fromagerie.

FRUITIÈRE DES AVANCHETS
La cave
La méthode

EXEMPLE DE GRAND PROJET · FROMAGERIE

Quatre fromages, une cave, une règle.

Un fromage est prêt si `joursAffinage >= 45`. Ce n'est pas le sujet imposé : c'est un exemple de domaine. Le vôtre n'est pas le système solaire.

En cave ce matin

Gruyère d'alpage

LAIT
vache

AFFINAGE
90 jours

PIÈCES
6

PRÊT À VENDRE EN STOCK

Tomme de chèvre

LAIT
chèvre

AFFINAGE
30 jours

PIÈCES
4

ENCORE EN CAVE EN STOCK

Raclette de printemps

LAIT
vache

AFFINAGE
60 jours

PIÈCES
0

PRÊT À VENDRE RUPTURE

Sérac du soir

LAIT
vache

AFFINAGE
12 jours

PIÈCES
8

ENCORE EN CAVE EN STOCK

Sur un téléphone de 390 pixels de large, les cartes passent sur une colonne.

FRUITIÈRE DES AVANCHETS
[La cave](#) [La méthode](#)

EXEMPLE DE GRAND PROJET · FROMAGERIE

Quatre fromages, une cave, une règle.

Un fromage est prêt si `joursAffinage >= 45`. Ce n'est pas le sujet imposé : c'est un exemple de domaine. Le vôtre n'est pas le système solaire.

En cave ce matin

Gruyère d'alpage

LAIT
vache

AFFINAGE
90 jours

PIÈCES
6

PRÊT À VENDRE **EN STOCK**

Tomme de chèvre

LAIT
chèvre

AFFINAGE

Vous créez exactement ceci

```
sas/  
├─ grand-projet/  
│  └─ index.html  
│     └─ css/  
│        └─ style.css  
│     └─ js/  
└─ votre-classe.js
```

Le départ contient `js/fromage.js` comme exemple. Si votre classe est `velo`, renommez-le `velo.js` et remplacez aussi le chemin dans la dernière balise `<script>` de `index.html`. Le HTML et le CSS forment déjà la page ; vous adaptez ses textes au lieu de les recopier depuis ce PDF.

Étape 1 — Dire le domaine avant le code

Imaginez une personne qui ouvre votre page. Que vient-elle chercher ? Quel type d'objet regarde-t-

elle ? Quelle question utile pose-t-elle à cet objet ?

Écrivez ces trois lignes sur papier ou dans vos notes :

```
Qui ouvre la page : ...  
Ma classe : ...  
Ma règle : ... retourne true quand ...
```

La règle doit pouvoir se dire en une phrase précise. Par exemple, la fromagerie montre la forme : « `estPret()` retourne `true` quand `joursAffinage` `>= 45`. » Vous gardez la forme, puis vous changez les mots pour votre sujet.

Étape 2 — Adapter le départ

Ouvrez `sas/grand-projet/index.html` par un double-clic. Vous voyez le chrome de la page : en-tête, titre, zone de cartes et zone d'explication. La fromagerie est un exemple provisoire ; les cartes sont absentes tant que la classe n'est pas terminée.

Dans `index.html`, remplacez les textes visibles par ceux de votre domaine. Ne changez pas les identifiants `liste` et `oral`, utilisés par le JavaScript. Renommez ensuite `js/fromage.js` avec le nom de votre classe, puis corrigez le chemin du `<script>`.

- ☐ Le titre annonce votre sujet.
- ☐ Le nom du fichier JS correspond à votre classe.
- ☐ La page charge ce fichier JS.
- ☐ Les dossiers correspondent exactement à l'arbre ci-dessus.

Étape 3 — Écrire le moule et une règle

Ouvrez votre fichier JS. Le squelette `fromage.js` montre un constructeur, deux méthodes à trous et la suite d'affichage. Une **classe** est le moule ; `new VotreClasse(...)` produit **un objet réel**, comme un moule à billets produit un billet réel.

Renommez la classe, ses propriétés et ses paramètres. Gardez seulement les propriétés dont votre règle a besoin. Complétez ensuite votre méthode principale avec la règle écrite à l'étape 1.

Pour l'exemple fromagerie, le trou serait remplacé par cette règle :

```
return this.joursAffinage >= 45
```

Dans votre projet, la condition doit employer vos propriétés. Ne recopiez pas `joursAffinage` si votre domaine parle de vélos, de livres ou de concerts.

Étape 4 — Parler à un seul objet

Avant une liste, fabriquez un seul objet sous la classe. Donnez-lui un nom singulier, puis appelez sa méthode dans la console.

La fromagerie du départ montre la forme :

```
const gruyere = new Fromage("Gruyère d'alpage", "vache", 90, 6);
gruyere.estPret();
```

Votre fichier doit avoir un objet de **votre** classe et un appel de **votre** règle. La console doit répondre `true` ou `false`, et vous devez expliquer ce résultat avec les valeurs de l'objet.

Étape 5 — Passer à trois objets

Vous savez maintenant questionner un objet seul. Créez une liste d'au moins trois objets du même domaine, avec des valeurs qui font répondre `true` à certains et `false` à d'autres.

Adaptez la boucle fournie dans le départ. Elle doit demander votre méthode à chaque objet, puis utiliser la réponse pour écrire la carte. Elle ne doit pas recopier la condition de votre règle.

Vérifiez deux cas à l'écran :

- un objet qui satisfait la règle ;
- un objet qui ne la satisfait pas.

Si les trois cartes répondent pareil, changez une donnée : votre page doit montrer que la règle décide réellement.

Étape 6 — Faire la preuve à l'oral

Montrez d'abord un `new VotreClasse(...)`, puis la méthode principale, puis la page. Sans lire le fichier, dites : « Cet objet représente... La méthode décide... Elle répond vrai quand... »

Votre interlocuteur doit pouvoir retrouver sur une carte les données qui expliquent la réponse. Deux minutes suffisent ; une longue visite du HTML ne prouve pas que la règle est comprise.

Réussi si

- [] Une personne comprend votre domaine en ouvrant la page.
- [] Votre classe, vos propriétés et vos mots ne sont plus ceux de la fromagerie.
- [] Vous avez testé un seul objet avant de créer la liste.
- [] Une règle nommée vit dans la classe et produit au moins un `true` et un `false`.
- [] La boucle demande la règle aux objets ; elle ne la recalcule pas.
- [] La page se lit à 1440 et à 390 pixels de large.

- [] Vous expliquez la règle sans lire.

AVEC L'IA

Après avoir choisi votre domaine et écrit la règle, montrez à une IA votre classe et un seul objet. Demandez :
« Posez-moi trois questions pour vérifier que je comprends pourquoi cette méthode appartient à l'objet. »
Répondez avec vos mots ; ne lui demandez pas d'inventer le domaine ni de générer le projet entier.

POUR LES RAPIDES

Ajoutez une deuxième méthode qui répond à une autre question du **même** domaine. Faites-la apparaître sur les cartes, puis expliquez pourquoi elle utilise les propriétés de l'objet. Ne démarrez pas un deuxième univers.

Livrable

Rendez le dossier `sas/grand-projet/`. À l'oral, montrez un objet, dites la règle sans lire et indiquez la ligne où la boucle demande cette règle à chaque objet.